

Summary of Improvements — AVALOQ

Hello AVALOQ team — and thank you!

It is a real pleasure to share this proposal with you. Every point below is a direct response to the questions and ideas raised during our review sessions, and we are committed to shaping each capability around the way **AVALOQ actually works** day to day.

For each new improvement we are happy to **walk you through how it will behave in practice** — with live demos, sample reports, and example scenarios — so there are no surprises before development starts.

We would also love to **schedule a dedicated working session with the AVALOQ team** to fine-tune every improvement to your exact needs: priorities, terminology, screens, exports, and integration points. Your feedback will drive the final design.

Looking forward to building this together. 🍷

Prepared by: Osvaldo Martini **Date:** 2026-05-05 **Scope:** Five capability improvements for the AR Web Scanner — two new core features and three productizations of existing strengths

Improvement 1 — Test Traceability (Requirements ↔ Test Coverage)

Status: Proposed **Kickoff:** 2026-05-06 **Final delivery:** 2026-06-05

What it is

A new layer that links each automated test (bot job) to one or more business **Requirements**, so you can answer at any moment:

- Which requirements are covered by tests?
- Which requirements are **not** covered?
- Which requirements **failed** in the last run?
- What is the overall coverage percentage?

Why it matters

Today the scanner runs tests and produces an execution report, but there is no link back to the requirements being validated. Stakeholders, auditors, and compliance teams cannot get a coverage view without manual spreadsheets.

What you will see

- A new **Requirements** screen where requirements can be created, edited, archived
- A new **Linked Requirements** tab inside each test job, to attach one or more requirements
- **CSV import** so existing requirement lists (from Jira, Azure DevOps, or any spreadsheet) can be loaded in bulk
- A new **Coverage** sheet inside the existing Excel execution report, showing each requirement with: status, last run date, last result (Pass / Fail / Mixed / Never run), and the list of jobs that cover it
- Color-coded rows (green / red / amber / grey) for quick visual scan
- A summary line at the top: *Total requirements / Covered / Passing / Coverage %*

Delivery plan

Step	What is delivered	Window	Due date
1	Foundations to store requirements and their links to tests	2026-05-06 → 2026-05-08	Fri 2026-05-08
2	Tests can carry their requirement links (saved + reloaded correctly)	2026-05-11 → 2026-05-12	Tue 2026-05-12
3	New Requirements screen — create, edit, archive, search	2026-05-13 → 2026-05-19	Tue 2026-05-19
4	Linking screen — attach requirements to a test job and to individual blocks	2026-05-20 → 2026-05-22	Fri 2026-05-22
5	CSV import for bulk loading of existing requirements	2026-05-25 → 2026-05-27	Wed 2026-05-27
6	Coverage sheet added to the Excel execution report	2026-05-28 → 2026-06-03	Wed 2026-06-03
7	End-to-end validation, user documentation, sign-off	2026-06-04 → 2026-06-05	Fri 2026-06-05

Out of scope for the first release

- Live two-way synchronization with Jira or Azure DevOps (planned for a later release)
- PDF or HTML coverage exports

- Step-by-step traceability inside a single test (only Job-level and Block-level for now)
 - Historical trend reports across many runs
-

Improvement 2 — Video Recording of Test Executions

Status: Proposed **Kickoff:** 2026-05-06 (in parallel with Improvement 1) **Final delivery:** 2026-06-08 (parallel) — or **2026-07-09** if delivered after Improvement 1

What it is

Every test run can now be recorded as a video file. When a test fails, you can open the recording and see exactly what was on screen at the moment of failure, instead of trying to reproduce the issue.

Why it matters

When a test fails today, the user has only a status line and possibly a single screenshot. To understand the failure they often need to re-run the test manually. A video recording removes that effort and gives a permanent visual evidence trail — useful for support, audit, and stakeholder review.

What you will see

- One **video file (.mp4)** automatically saved for each test run, organized by job name and date/time
- **Click-to-seek** links inside the existing Excel execution report — clicking a row opens the video at the exact moment that step was executed
- A **failure-only mode**: keeps a rolling buffer of the last few seconds and saves it permanently only when something fails (avoids large files when everything passes)
- A **Settings screen** where the user can switch recording On / Off / Failure-only, choose quality, and choose to record only the browser window or the full screen
- Files stored locally in a clearly named folder, ready for archival or attachment to a support ticket

Delivery plan

Step	What is delivered	Window	Due date
1	Initial recording capability proven on a sample run	2026-05-06 → 2026-05-08	Fri 2026-05-08
2	Recording engine integrated into the application (start / stop / step markers)	2026-05-11 → 2026-05-14	Thu 2026-05-14
3	Pre-Launch Test runs are automatically recorded	2026-05-15 → 2026-05-18	Mon 2026-05-18
4	Settings screen to control recording (On / Off / Failure-only, quality, region)	2026-05-19 → 2026-05-21	Thu 2026-05-21
5	Final storage layout — clean folder structure and file naming	2026-05-22 → 2026-05-25	Mon 2026-05-25
6	Failure-only mode: short video clip captured automatically on failure	2026-05-26 → 2026-05-28	Thu 2026-05-28
7	Click-to-seek links from the Excel execution report into the video	2026-05-29 → 2026-06-04	Thu 2026-06-04
8	End-to-end validation, user documentation, sign-off	2026-06-05 → 2026-06-08	Mon 2026-06-08

Out of scope for the first release

- Drawing annotations on the video (highlighting clicked elements)
- Cloud upload of recordings
- Embedded video player inside the application
- Support for operating systems other than Windows

Combined timeline

Scenario	Improvement 1	Improvement 2	Both done by
Parallel (recommended)	2026-06-05	2026-06-08	2026-06-08 (Mon)
Sequential (RTM first)	2026-06-05	2026-07-09	2026-07-09 (Thu)

The two improvements are independent and can be delivered in parallel without conflict.

Summary of business benefits

Benefit	From Improvement 1	From Improvement 2
Coverage visibility for stakeholders	✓	—
Audit / compliance evidence	✓	✓
Faster failure diagnosis	—	✓
Reduced manual reproduction effort	—	✓
Bulk onboarding of existing requirements	✓	—
Visual proof of test execution	—	✓
Single Excel report tying requirements + execution + recording together	✓	✓

Both improvements together turn the scanner from a *test authoring tool* into a *full test management and evidence platform* — the two missing pieces most often raised by stakeholders.

Other Questions raised by email (reviewed in the ARWeb presentation)

Question 1 — Customizable element identification

"It was noticed that ARWeb has a default rendering of the elements. Is this customizable, or can it be updated if we want to use specific elements as identifier?"

Answer: Yes — fully customizable today.

ARWeb already supports a **per-site Priority Table** that defines which attribute is preferred to identify each element (for example: a custom `test-id`, an `aria-label`, an internal data attribute, the visible text, the position, etc.). The order is configurable, and different sites can have different priority orders.

Status: ✓ Already available. Configuration is done in the site profile.

Possible enhancement (Improvement 3 — Visual Priority Editor): A small UI that lets the user reorder identifier priorities with drag-and-drop instead of editing the configuration file by hand.

Step	What is delivered	Effort
1	Visual drag-and-drop priority editor inside the configuration screen	3 working days
2	Per-site profile selector + live preview of which attribute would be picked on the current page	2 working days
3	Validation + documentation	1 working day
Total	6 working days	~1.5 weeks

Suggested window: 2026-06-09 → 2026-06-16

Question 2 — Post-conditions and recovery steps on failure

"Is there a way to force post-conditions and recovery steps in case of test failure?"

Answer:  Already covered.

ARWeb already supports **conditional flow control** inside a bot job:

- **If / Else conditions** — branch the test depending on what is on the page
- **Loop conditions** — retry a step until it succeeds or a maximum is reached
- **Check Value** — explicit assertion that the page state matches the expected value, with a defined fallback path

These can be combined to model "try → on failure → run recovery steps → continue / abort", which is exactly what post-conditions and recovery flows require.

Status:  Already available. Demonstrated in the ARWeb presentation.

Optional polishing (Improvement 4 — Recovery Templates): Pre-built, reusable recovery templates (e.g., "close popup and retry", "reload page and resume", "logout and re-login") so users do not have to rebuild the same recovery flow on every job.

Step	What is delivered	Effort
1	Recovery template library (5 standard templates)	3 working days
2	"Insert recovery template" action in the job editor	2 working days
3	User-saved custom templates	2 working days
4	Validation + documentation	1 working day
Total	8 working days	~2 weeks

Suggested window: 2026-06-17 → 2026-06-26

Question 3 — Storing values from elements that look like buttons

"Unable to store the value of position quantity because it is being identified as a button. With this, we cannot proceed with verification / checking of the value."

Answer:  Already supported through **Dynamic Elements**, and we can extend it further.

ARWeb already creates **Dynamic Elements** for a bot job. A Dynamic Element lets the bot understand whether a given web element is *really* a button, an input, a label, or a value carrier — independently of how the page tags it. This means a "button" that actually displays a quantity can be re-classified as a value carrier and its content can be read, stored, and verified like any other field.

The same Dynamic Element mechanism can also **inject elements into the page** at run time to simulate scenarios that the page does not natively offer — for example:

- Inject a text area that accepts a **signature** drawn by the user
- Inject a field that accepts **images** or attachments
- Inject a temporary value carrier so the bot can capture and compare data that is otherwise locked inside non-standard widgets

Status:  Already supported. Recommended to package this as a named feature so customers know it exists.

Suggested productization (Improvement 5 — Visual Field Mapping & Dynamic Elements): A new visual feature where the user points at any region on the page (WYSIWYG selection / OCR-assisted mapping) and chooses what kind of element it should behave as: *value*, *input*, *signature pad*, *image upload*, etc. The bot then treats that region accordingly during execution and verification.

Step	What is delivered	Effort
1	Visual region picker (point at any area of the page and select it)	4 working days
2	Type chooser — Value / Input / Signature / Image / Custom	2 working days
3	OCR-assisted mapping — automatic value reading from the picked region	3 working days
4	Injected widgets — signature pad, image uploader	4 working days
5	Save mapping as part of the job (reusable across runs)	2 working days
6	Verification action — assert value of a mapped region	2 working days
7	Validation + documentation	2 working days
Total	19 working days	~4 weeks

Suggested window: 2026-06-29 → 2026-07-24

Updated combined roadmap (all five improvements)

Improvement	Status	Effort	Suggested window	Final delivery
1 — Test Traceability (RTM)	Proposed	~22 days	2026-05-06 → 2026-06-05	2026-06-05
2 — Video Recording	Proposed	~23 days	2026-05-06 → 2026-06-08 (parallel)	2026-06-08
3 — Visual Priority Editor	Optional enhancement	~6 days	2026-06-09 → 2026-06-16	2026-06-16
4 — Recovery Templates	Optional polishing	~8 days	2026-06-17 → 2026-06-26	2026-06-26
5 — Visual Field Mapping & Dynamic Elements	Proposed (high value)	~19 days	2026-06-29 → 2026-07-24	2026-07-24

End-to-end delivery of all five improvements: 2026-07-24 (Fri) — assuming Improvements 1 and 2 run in parallel and 3 → 4 → 5 run sequentially after them.

If Improvement 5 is prioritized (it answers the "value-vs-button" issue, the most operational pain point), it can be pulled forward to start in parallel with Improvement 2, bringing its delivery to mid-June.

